

SafeSight

Industrial PPE & Hazard Compliance Management System

D4 — Detailed Design Document

Team STARK

Member Names: Burak Kaygusuzoğlu – Nevzat Berk Akkaş – Ediz Akin

Course: System Modelling

Instructor: Edmond JAJAGA

1. Introduction

Purpose of this document: This D4 Detailed Design Document specifies the implementation-ready design of the SafeSight Incident Management Subsystem. It builds on the functional requirements defined in D2 (FR-05 to FR-12) and the three-tier layered architecture established in D3, adding module structure, design classes, API contracts, data schema, and interaction sequences sufficient for a developer to begin implementation without further design input.

Scope: This document covers only the Incident Management Subsystem — the operational core of SafeSight through which every PPE violation or hazard alert is transformed into a structured, validated, acted-upon, and audited incident record. The incident lifecycle governed here is: Alert → Incident → Validation → Action → Closure.

2. Subsystem Overview

The Incident Management Subsystem manages five sequential lifecycle phases: (1) Alert receipt and triage, (2) Incident creation and classification, (3) Safety Officer validation or rejection, (4) Corrective Action assignment to a Supervisor, and (5) Closure with audit trail generation. No phase may be skipped. The subsystem is the convergence point for data from the Monitoring, Reporting, and User Management subsystems.

Supported Functional Requirements (from D2):

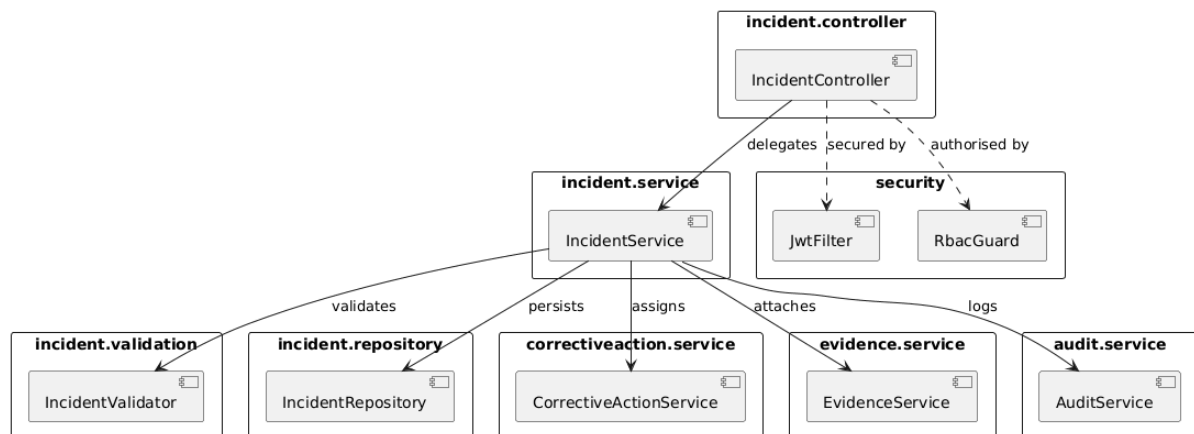
- FR-05 — Create an incident record from a validated PPE alert (zone, timestamp, severity, worker).
- FR-06 — Allow a Safety Officer to validate or reject an incident within a defined time window.
- FR-07 — Assign corrective actions to a Supervisor with due date and priority.
- FR-08 — Enforce lifecycle status transitions: OPEN → VALIDATED/REJECTED → IN_PROGRESS → CLOSED.
- FR-11 — Attach evidence (images, sensor data, video frames) to an incident at creation time.
- FR-12 — Generate an immutable audit log entry for every state transition and administrative action.

3. Module / Package Structure

The subsystem is organised into eight packages, each with a single responsibility. Business logic never touches the database directly; the controller never contains rules; validation is a standalone unit for testability.

Package	Responsibility
---------	----------------

incident.controller	RESTful HTTP endpoints. Parses requests, delegates to service, formats responses.
incident.service	Orchestrates lifecycle rules. Calls validation, repository, evidence, audit, and action services.
incident.validation	Input and business-rule validators. Enforces field types, enums, role constraints, and transition matrix.
incident.repository	ORM-abstracted CRUD for Incident and Alert. Query methods by status, zone, date, severity.
correctiveaction.service	Creates, assigns, tracks, and closes corrective actions. Notifies supervisors on assignment.
evidence.service	Manages evidence upload references and retrieval. Associates artefacts with incidents.
audit.service	Appends an immutable audit record for every lifecycle mutation (actor, timestamp, from/to state).
security	JWT authentication and RBAC middleware. Roles: SAFETY_OFFICER, SUPERVISOR, ADMIN.



4. Design Class Diagram

- IncidentController — methods: createIncident(req), validateIncident(id, req), assignAction(id, req). Dependency → IncidentService.
- IncidentService — methods: create(dto), validate(id, dto), escalate(id), close(id). Dependencies → IncidentRepository, EvidenceService, AuditService, CorrectiveActionService, IncidentValidator.
- IncidentValidator — methods: validateCreatedDTO(dto), validateTransition(current, next, role).
- IncidentRepository — methods: save(incident), findById(id), findByStatus(status), findByZone(zoneId).
- Incident (Entity) — id, alertId, zoneId, severity:enum, status:enum, reportedAt, closedAt, assignedTo. Associations: Evidence(1..*), CorrectiveAction(0..*), AuditLog(1..*).
- Alert (Entity) — id, detectedAt, zoneId, type, cameraId, confidence. Association → Incident(1..1).
- User (Entity) — id, name, role:enum{SAFETY_OFFICER|SUPERVISOR|ADMIN}, department.
- Evidence (Entity) — id, incidentId, type:enum, storageRef, uploadedAt.
- CorrectiveAction (Entity) — id, incidentId, assignedTo, description, priority:enum, dueDate, status:enum.
- EvidenceService — attach(incidentId, files), retrieve(incidentId).
- AuditService — log(actorId, incidentId, from, to, timestamp).
- CorrectiveActionService — assign(incidentId, dto), updateStatus(actionId, status), close(actionId).

Design classes differ from conceptual classes (D2/D3) in that they carry method signatures with typed parameters, ORM annotations, status-transition enforcement logic, and navigable associations with persistence-driven multiplicities. The IncidentStatus enum and transition matrix are implementation constructs absent from the conceptual model.

5. API Specifications

5.1 Create Incident — POST /api/v1/incidents

Field	Detail
Authorised Roles	SAFETY_OFFICER, ADMIN
Inputs	alertId (UUID, req); severity (LOW MEDIUM HIGH CRITICAL, req); zoneId (UUID, req); description (string ≤1000, opt); attachments (file refs[], opt)

Success Output	HTTP 201 — IncidentDTO { id, status:"OPEN", severity, zoneId, alertId, reportedAt, reportedBy }
Validation Rules	alertId must exist and be unlinked. zoneId must be active. One alert → at most one incident.
Error Cases	400 missing/invalid fields 404 alertId/zoneId not found 409 incident already exists for alertId 403 insufficient role

5.2 Validate Incident — PATCH /api/v1/incidents/{id}/validate

Field	Detail
Authorised Roles	SAFETY_OFFICER only
Inputs	id (path, UUID); decision (VALIDATED REJECTED, req); notes (string ≤500, required if REJECTED). validatedBy injected from JWT.
Success Output	HTTP 200 — updated IncidentDTO with status = VALIDATED or REJECTED, validatedAt timestamp.
Validation Rules	Incident must be OPEN. Validation window ≤ 24 h from reportedAt. notes mandatory on REJECTED.
Error Cases	400 missing decision/notes 404 id not found 409 not in OPEN status 410 window expired 403 not SAFETY_OFFICER

5.3 Assign Corrective Action — POST /api/v1/incidents/{id}/actions

Field	Detail
Authorised Roles	SAFETY_OFFICER, ADMIN
Inputs	id (path, UUID, must be VALIDATED); supervisorId (UUID, req); description (string ≤1000, req); priority (LOW MEDIUM HIGH, req); dueDate (ISO 8601 future date, req)
Success Output	HTTP 201 — CorrectiveActionDTO { actionId, incidentId, supervisorId, priority, dueDate, status:"ASSIGNED" }. Incident status → IN_PROGRESS.
Validation Rules	Incident must be VALIDATED. supervisorId must be an active SUPERVISOR. dueDate must be future.
Error Cases	400 missing fields / past dueDate / invalid enum 404 incident or supervisorId not found 409 incident not VALIDATED 403 insufficient role

6. Data Persistence View

A relational database is used because data is structured, multi-table transactions are required (e.g., simultaneously creating a CorrectiveAction record and advancing the incident status), and referential integrity is safety-critical. Evidence binaries are stored in object storage (S3-compatible); only the URI is persisted in the evidence table. The audit_logs table is append-only — no UPDATE or DELETE is issued against it by any service.

Table	Primary Key	Key Fields	Purpose
users	id (UUID)	name, role(enum), department, email, is_active	All system actors; role drives RBAC.
zones	id (UUID)	name, building, floor, is_active	Physical zones where incidents originate.
alerts	id (UUID)	detected_at, zone_id(FK), type, confidence, camera_id, linked_incident_id(FK nullable)	Raw detections; linked to at most one incident.
incidents	id (UUID)	alert_id(FK), zone_id(FK), severity(enum), status(enum), reported_at, reported_by(FK), validated_by(FK), closed_at	Core record. status enforces lifecycle.
evidence	id (UUID)	incident_id(FK), type(enum), storage_ref, uploaded_at, uploaded_by(FK)	Evidence references. Binary content in object storage.
corrective_actions	id (UUID)	incident_id(FK), assigned_to(FK), priority(enum), due_date, status(enum), completed_at	One incident may have multiple actions.
audit_logs	id (UUID)	incident_id(FK), actor_id(FK), action_type, from_status, to_status, timestamp, notes	Immutable append-only audit trail.

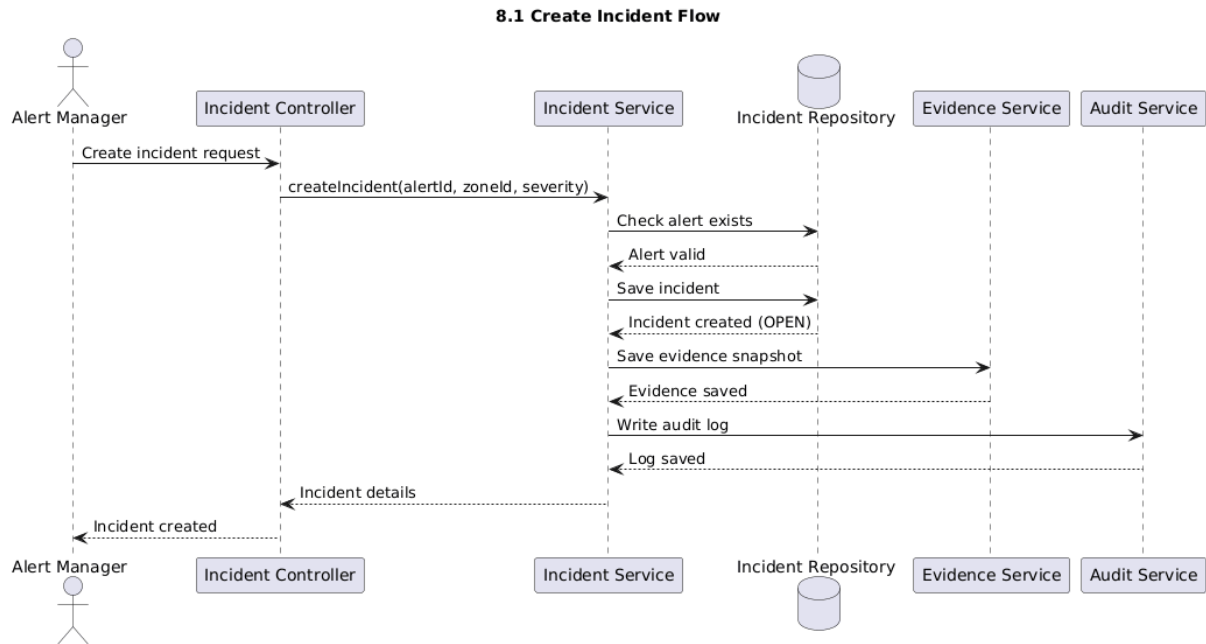
7. Edge Cases and Exception Handling

ID	Scenario	Handling Strategy
EC-01	Duplicate incident creation (same alertId, concurrent requests)	Unique DB constraint on alert_id. Second request receives HTTP 409 Conflict. No duplicate persisted.
EC-02	Validation window expired (Safety Officer acts after 24 h)	IncidentValidator checks reportedAt + window vs. now(). Returns HTTP 410 Gone. Incident auto-escalated to Admin.
EC-03	Invalid status transition (e.g., OPEN → CLOSED directly)	Transition matrix in IncidentValidator rejects illegal jump. HTTP 409 Conflict. No state change persisted.
EC-04	Evidence upload failure during incident creation	Transaction rolls back. HTTP 503. No partial incident record remains.
EC-05	Assigned Supervisor account deactivated after assignment	Action flagged UNATTENDED. Admin notified to reassign. Incident status unaffected. Historical reference preserved for audit.

EC-06	Corrective action due date exceeded without completion	Scheduled job flags action OVERDUE. Escalation notification dispatched to Supervisor and Safety Officer. Incident → ESCALATED.
EC-07	Concurrent validation by two Safety Officers	Optimistic lock (version field on incidents table). One transaction commits; second receives HTTP 409 Conflict.

8. Sequence Diagrams

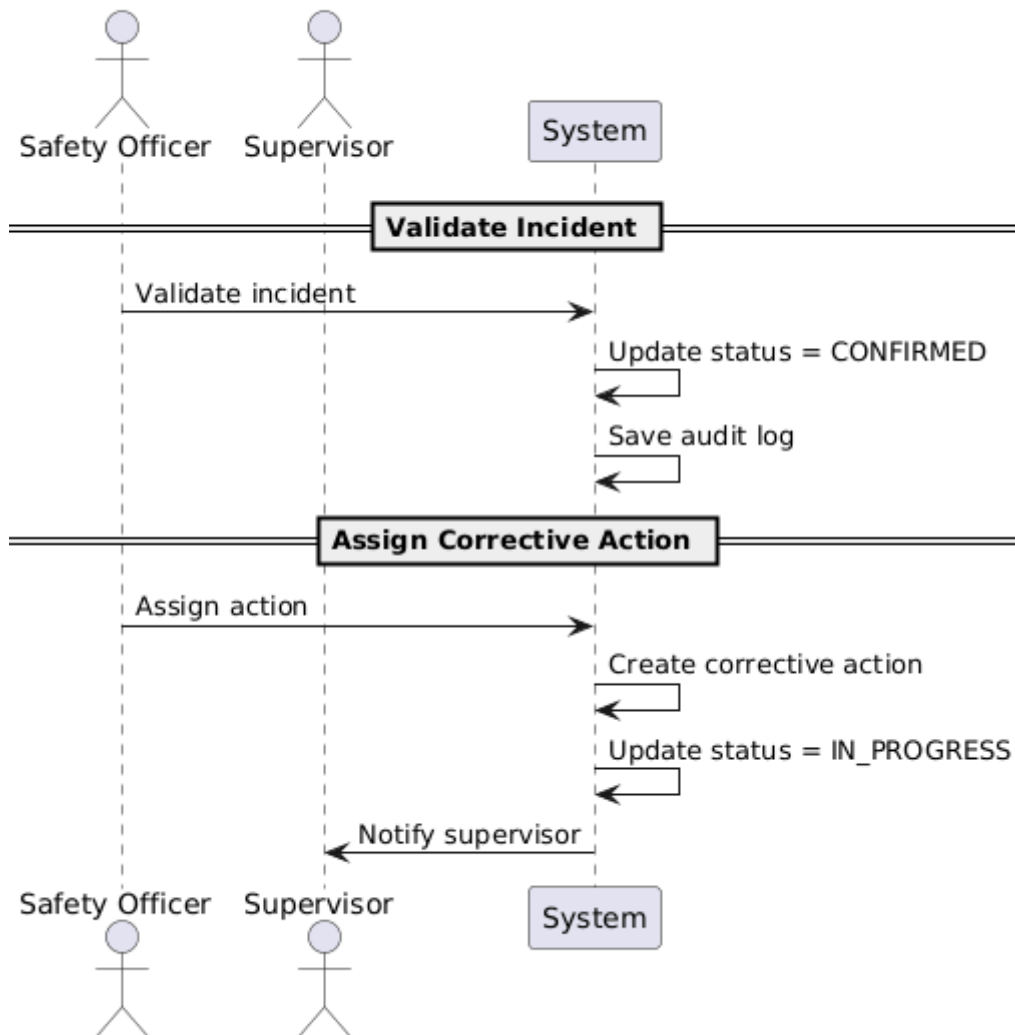
8.1 Create Incident Flow



1. AlertManager → IncidentController: POST /api/v1/incidents (alertId, severity, zoneId, attachments)
 2. IncidentController → IncidentValidator: validateCreateDTO(dto) — throws on invalid input
 3. IncidentController → IncidentService: create(dto)
 4. IncidentService → IncidentRepository: findAlertById(alertId) — confirm exists and unlinked
 5. IncidentService → IncidentRepository: save(newIncident) [status = OPEN]
 6. IncidentService → EvidenceService: attach(incidentId, attachments) [opt — if attachments present]
 7. IncidentService → AuditService: log(actorId, incidentId, null, OPEN, now())
 8. IncidentController → AlertManager: HTTP 201 Created + IncidentDTO
- [alt] If alertId already linked → IncidentRepository returns conflict → HTTP 409 Conflict

8.2 Validation + Corrective Action Flow

8.2 Validate Incident & Assign Action (Simple)



PART A — Validation:

1. SafetyOfficer → BrowserUI: submit validate form (decision, notes)
2. BrowserUI → IncidentController: PATCH /api/v1/incidents/{id}/validate
3. IncidentController → IncidentValidator: validateTransition(OPEN, VALIDATED, SAFETY_OFFICER)
[opt] If window expired → HTTP 410 Gone
4. IncidentController → IncidentService: validate(id, dto)
5. IncidentService → IncidentRepository: updateStatus(id, VALIDATED, validatedBy, validatedAt)
6. IncidentService → AuditService: log(actorId, id, OPEN, VALIDATED, now())
7. IncidentController → BrowserUI: HTTP 200 OK + IncidentDTO

PART B — Corrective Action Assignment:

8. SafetyOfficer → BrowserUI: submit assign-action form (supervisorId, description, priority, dueDate)
9. BrowserUI → IncidentController: POST /api/v1/incidents/{id}/actions
10. IncidentService → IncidentValidator: validateTransition(VAIDATED, IN_PROGRESS, SAFETY_OFFICER)
11. IncidentService → CorrectiveActionService: assign(incidentId, dto)
12. CorrectiveActionService → IncidentRepository: save(newCorrectiveAction)
13. IncidentService → IncidentRepository: updateStatus(id, IN_PROGRESS)
14. IncidentService → AuditService: log(actorId, id, VAIDATED, IN_PROGRESS, now())
15. CorrectiveActionService → Supervisor: sendNotification(supervisorId, actionDetails)
16. IncidentController → BrowserUI: HTTP 201 Created + CorrectiveActionDTO

Sequence diagrams are used because the subsystem's value lies in component choreography under temporal constraints. They specify message ordering and guard conditions (alt/opt fragments) that a class diagram cannot express — critical for a safety workflow where audit logging must fire after persistence commits, not before.

9. UML Artefact Summary

Diagram	Section	What It Covers
Design Class Diagram	§4	All design classes: controller, service, validator, repository, entities. Typed attributes, method signatures, associations with multiplicities. Sufficient for skeleton code generation.
Sequence Diagram 1 — Create Incident	§8.1	Message flow for incident creation across 7 components. Includes alt fragment for duplicate alertId.
Sequence Diagram 2 — Validate + Assign	§8.2	Two-part flow: validation by Safety Officer + corrective action assignment to Supervisor. Includes opt fragment for expired window.

No additional diagrams are required: the package table (§3) replaces a component diagram; the API tables (§5) replace an interface diagram; the schema table (§6) replaces an ER diagram.

10. Requirements Traceability Matrix

Maps each D2 Functional Requirement served by this subsystem to its primary design element and the artefacts in this document where it is realised.

FR	Description	Primary Design Element	Document Artefacts
FR-05	Create incident from validated PPE alert	IncidentService.create()	§5.1 API, §8.1 Sequence Diagram, §6 incidents table
FR-06	Safety Officer validates or rejects incident	IncidentValidator.validateTransition()	§5.2 API, §8.2 Sequence Diagram (Part A)
FR-07	Assign corrective action to Supervisor	CorrectiveActionService.assign()	§5.3 API, §8.2 Sequence Diagram (Part B), §6 corrective_actions table
FR-08	Enforce lifecycle status transitions	IncidentValidator (transition matrix)	§4 Class Diagram, §7 EC-02/EC-03, §6 incidents.status constraint
FR-10	Role-based access control on all operations	security package (JWT + RBAC middleware)	§3 security package, §5 Authorised Roles rows, §4 User.role enum
FR-11	Attach evidence to incident at creation	EvidenceService.attach()	§5.1 attachments field, §8.1 step 6, §6 evidence table
FR-12	Immutable audit log for every state transition	AuditService.log()	§8.1 step 7, §8.2 steps 6+14, §6 audit_logs table

Note: FR-09 (real-time dashboard notifications) is handled by the Notification Subsystem and is not within the scope of this document. All seven requirements listed above are satisfied by named design elements within this subsystem. FR-10 is enforced at the controller boundary via middleware, not embedded in individual services. FR-12 is the final call in every IncidentService method that mutates state, guaranteeing no transition is unlogged.